

0.91.2. Le titre, puis le jeu — le score et rejouer

Le 0.91.1, et sur l'écran FIN : le score de la partie, et START pour rejouer. Tout se remet au départ : une fonction `nouvellePartie()`.

1. Ce qu'il faut comprendre

C'est le 0.91.1, avec une chose en plus : l'écran de fin montre le **score**, et **START** relance une partie.

Ce qui est nouveau ici : tout remettre au départ. Une nouvelle partie, c'est le A en (5, 8), le P en (15, 8), le score à 0, le temps à 30, et l'écran du jeu redessiné. Ces lignes servent **deux fois** : depuis le titre, et depuis la fin. On les range dans `nouvellePartie()` (le 0.90.6).

Remettre les variables, c'est indispensable : sans `score = 0`, la deuxième partie commencerait avec le score de la première ; sans `px = 15`, le P resterait hors de l'écran (en 20), et on ne pourrait plus le ramasser.

La fin mène au jeu, pas au titre (le 0.90.7) : START encore enfoncé ferait sauter le titre aussitôt.

La première seconde de la nouvelle partie est un peu courte : le chronomètre de `chaque(1000)` a continué de tourner pendant l'écran de fin. Au retour dans le jeu, il est déjà « à l'heure » : 30 devient 29 tout de suite.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <couleurTexte>` → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2114 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          97 o ← la console, à l'allumage — la mise en route de la console
texte_grand_0()    195 o ← main() — ta fonction
chaque_minuteur()  124 o ← main() — une fonction fournie par le langage
deplace_croix()    217 o ← main() — une fonction fournie par le langage
viderEcran()       33 o ← nouvellePartie(), main() — ta fonction
nouvellePartie()   66 o ← main() — ta fonction
main()             239 o ← démarrage — ta fonction
VBlank             8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
```

AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre	– attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer()	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer()	– efface des cases
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre()	– écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	848 o	← CopierTuiles	– la police entière, et 9 tuiles dessinées
"SCORE"	5 o	← texte()	– un texte gravé
"TEMPS"	5 o	← texte()	– un texte gravé
"APPUIE SUR START"	16 o	← texte()	– un texte gravé
"FIN"	3 o	← texte()	– un texte gravé
"SCORE"	5 o	← texte()	– un texte gravé
"START : REJOUER"	15 o	← texte()	– un texte gravé
GRAND_TABLE_0_0_0	12 o	← poser()	– une table gravée
GRAND_TABLE_0_1_1	12 o	← poser()	– une table gravée
GRAND_TABLE_0_2_2	12 o	← poser()	– une table gravée
GRAND_TABLE_0_3_3	12 o	← poser()	– une table gravée

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 6	148 o sur place, dans texte_grand_0(), deplace_croix(), nouvellePartie() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (12 o), GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)
texte()	× 6	66 o sur place, dans nouvellePartie(), main() + EcrireTexte (10 o)
bouton()	× 6	30 o sur place, dans deplace_croix(), main() + LireManette (37 o)
nombre()	× 3	30 o sur place, dans main() + EcrireNombre (33 o)
effacer()	× 2	29 o sur place, dans deplace_croix(), viderEcran() + AdresseCase (10 o), EffacerCases (12 o)
couleurFond()	× 1	12 o sur place, dans main()
images()	× 2	4 o sur place, dans chaque_minuteur(), deplace_croix()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

deplace_croix()	× 1	19 o sur place, dans main()
chaque_minuteur()	× 1	10 o sur place, dans main()
viderEcran()	× 2	6 o sur place, dans nouvellePartie(), main()
nouvellePartie()	× 2	6 o sur place, dans main()
texte_grand_0()	× 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <effacer>           // efface des cases, ou tout le fond
#include <poser>              // pose une tuile sur une case du fond
#include <ALPHABET>           // les lettres de la police : ALPHABET[0] est le A
#include <texte>              // écrit un texte à l'écran
#include <couleurTexte>       // choisit la couleur des lettres
#include <texteGrand>         // écrit un texte agrandi, de 1 à 20 fois
#include <bouton>             // lit un bouton de la manette
#include <deplace_croix>      // une tuile qui suit la croix, case par case
#include <nombre>             // écrit un nombre en chiffres
#include <chaque>             // répond 1 toutes les n millisecondes
```

2. UNE VARIABLE — ECRAN

```
uint8_t ecran = 0;           // 0 = le TITRE, 1 = le JEU, 2 = la FIN
```

3. UNE VARIABLE — X

```
uint8_t x = 5;               // le A : sa colonne...
```

4. UNE VARIABLE — Y

```
uint8_t y = 8;           // ...et sa ligne (le 0.81)
```

5. UNE VARIABLE — PX

```
uint8_t px = 15;         // la pièce, le P : sa colonne...
```

6. UNE VARIABLE — PY

```
uint8_t py = 8;          // ...et sa ligne
```

7. UNE VARIABLE — SCORE

```
uint8_t score = 0;        // les pièces ramassées
```

8. UNE VARIABLE — TEMPS

```
uint8_t temps = 30;       // les secondes qui restent
```

9. UNE FONCTION — VIDERECRAN()

```
void viderEcran() {  
    for (uint8_t l = 0; l < 18; l++) {  
        effacer(0, l, 20);  
    }  
}  
  
// ---- NOUVEAU : une partie qui commence, TOUT au départ ----  
// Depuis le titre, ET depuis la fin (START : rejouer).
```

10. UNE FONCTION — NOUVELLEPARTIE()

```
void nouvellePartie() {  
    x = 5;                      // le A revient à sa place  
    y = 8;  
    px = 15;                    // le P aussi (il était peut-être en 20, ramassé)  
    py = 8;  
    score = 0;                  // pas encore de pièce  
    temps = 30;                 // 30 secondes de nouveau  
    ecran = 1;                  // on est sur le JEU  
    viderEcran();  
    poser(px, py, ALPHABET[15]);  
    texte(0, 17, "SCORE");  
    texte(11, 17, "TEMPS");  
}
```

11. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurTexte(31, 16, 0);
    texteGrand(4, 5, "JEU", 4);
    texte(2, 12, "APPUIE SUR START");

    while (true) {
        image();

        if (ecran == 0 && bouton(START)) {
            nouvellePartie();          // la première partie
        }

        if (ecran == 1) {
            deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
            if (x == px && y == py) {               // le A sur le P : ramassée (le 0.81)
                score = score + 1;
                px = 20;                            // la pièce part hors de l'écran
            }
            nombre(6, 17, score);
            nombre(17, 17, temps);
        }

        if (ecran == 1 && chaque(1000)) {
            temps = temps - 1;
            if (temps == 0) {
                ecran = 2;
                viderEcran();
                texte(8, 6, "FIN");
                texte(5, 9, "SCORE");           // NOUVEAU : le score de la partie
                nombre(11, 9, score);
                texte(2, 13, "START : REJOUER");
            }
        }

        // ---- NOUVEAU : sur la fin, START relance une partie.
        if (ecran == 2 && bouton(START)) {
            nouvellePartie();          // la même fonction : tout au départ
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



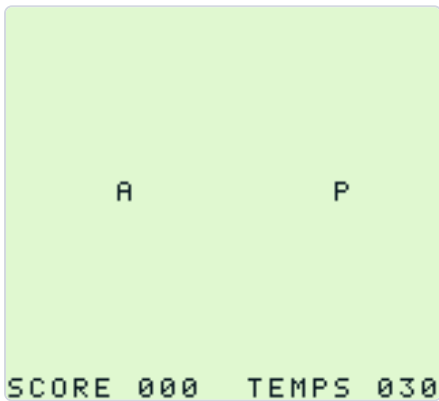
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



START — l'écran a changé

5. Ce qu'on doit voir

Une partie de 30 secondes ; à la fin : FIN, le score, START : REJOUER. START : une nouvelle partie, score 0, temps 30.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la fin montre le score : 001
- ✓ START : une nouvelle partie (écran 1, score 0, temps 30 ou 29) (temps = 29)
- ✓ le P est revenu en (15, 8), et le A en (5, 8)

Ensuite : **0.91.3. Le titre, puis le jeu — un cadre de murs X** — chapitre 0 : Avant tout. · [gameboy3](#) — écrire une cartouche Game Boy en C++